

Distributed ETL Pipeline for Public Transit Data Using Apache Airflow

FINAL REPORT
Version 1.0

CS 4850-01
Fall 2025
12/9/2025
Professor Sharon Perry

SP-112 ETL Pipeline

Website - <https://cs4850-seniorproject-yorkarunahavivcook.github.io/Development/>

GitHub - <https://github.com/CS4850-SeniorProject-YorkArunaHavivCook/Development>

STATS AND STATUS	
LOC	2400
Components/Tools	3 – Python, Apache Airflow, DuckDB
Hours Estimate	201
Hours Actual	395
Status	Project is 100% complete

Revision History

Name	Date	Reason For Changes	Version
Development	Nov. 30		V0

Appendix	Dec. 5	Added more designs for document	V1
----------	--------	---------------------------------	----

Contents

- Introduction 4
- Requirements 4
 - Functional Requirements 4
 - Non-Functional Requirements 5
- Analysis/Design 6
 - Module 1: Data Retrieval 6
 - Module 2: Data Processing 7
 - Module 3: Data Storage 7
 - Scheduling: Apache Airflow 8
- Development 8
- Database Connection 9
 - Architecture Overview 9
 - GTFSProcessor 10
 - GTFSTManager 10
- 1. Test Objectives 10
- 2. Scope of Testing 11
- 3. Test Cases 11
 - 3.1 Static file retrieval 11
 - 3.2 Static data saved 11
 - 3.3 Realtime files retrieved 11
 - 3.4 Realtime data saved 11
 - 3.5 Realtime data processed 11
 - 3.6 Airflow managing tasks 12
- 4. Test Procedures 12
 - 4.1 Static file retrieval 12
 - 4.2 Static data saved 12

4.3 Realtime files retrieved	12
4.4 Realtime data saved	12
4.5 Realtime data processed	12
4.6 Airflow managing tasks	12
Test Environment	13
Test Data	13
Software Test Report (STR)	14
Conclusion	14
Appendix	15

Introduction

Public transit systems produce massive streams of real-time data, including schedules, delays, and vehicle positions. Without a structured ETL pipeline, this data remains fragmented and difficult to process, limiting the ability to extract actionable insights for improving transit efficiency and rider experience. ETL pipeline is a raw data processing tool that extracts raw source data, transforms it into a structure format, and loads it into a database for analysis.

Our project addresses this challenge by implementing an ETL pipeline using Apache Airflow, DuckDB, and Python to process MARTA's GTFS (Global Transit Feed Specification) data. As a result, the pipeline gives further insight into transit operations showcasing patterns in schedule routes, identifying delays, and making data-driven decisions for schedule optimization.

Requirements

Functional Requirements

1.1 Extract GTFS data from MARTA

- GTFS Static
- GTFS Realtime
- All data retrieval, cleaning, and storing will be orchestrated by Apache Airflow DAGs
- Real-time data retrieval will occur hourly
- Logs of DAGs kept for viewing errors, retries, etc.

1.2 Transform the data into usable format

- Handle missing data
- Process incorrect/questionable data points

1.3 Load data into DuckDB database

- Relational database that joins GTFS static data with real-time route information

1.4 Have access to database of data from the MARTA GTFS feed

- Database stores cleaned data
- Data can be queried for later analysis

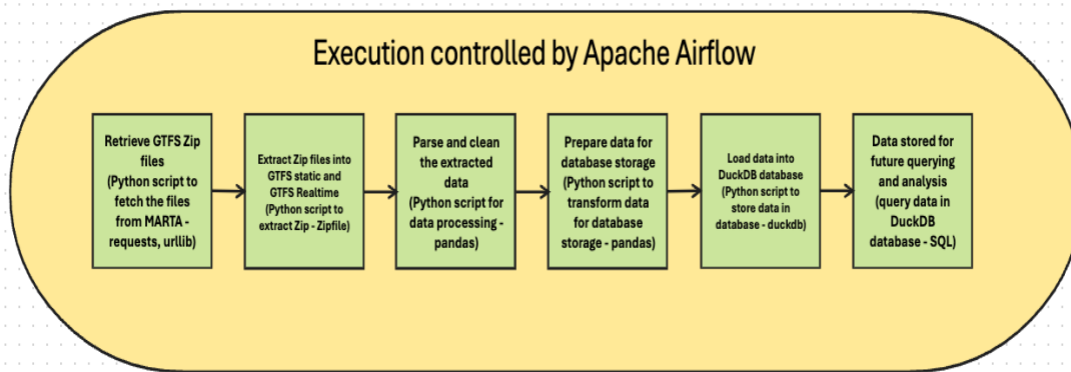
1.5 Error handling

- System will be able to handle errors such as failed data transfers and missing data
- System will show visible alert if an error is detected that requires user attention

Non-Functional Requirements

- ETL Pipeline data will be accurate and complete. Missing/incomplete data will be handled during the “Transform” part of the ETL cycle.
- Resources should be properly allocated to balance the workload of the tasks.
- The ETL Pipeline must be able to sustain strong scalability going through the large database

Analysis/Design



Module 1: Data Retrieval

- Classification: Python script / subsystem within Airflow DAG
- Definition: Retrieves GTFS static and real-time data (zip files via API) and extracts contents. Handles both recurring hourly polling and one-time setup of static schedule files.
- Constraints:
 - API availability
 - Scheduled maintenance
 - Network bandwidth
 - Data saved locally in raw GTFS format
 - Task must be completed within Airflow DAG execution window (Hourly)
- Resources:
 - Network connection
 - Python libraries (requests, urllib, zipfile, GTFS, DuckDB)
 - Disk storage on VM (temporary staging area for raw files).
 - CPU/memory usage is low; I/O bound to network and disk.
- Interface/Exports:
 - Input: API endpoints for GTFS data

- Output: Extracted .txt and Protobuffer GTFS files ready for processing
- Exported to Module 2 (Data Processing)

Module 2: Data Processing

- Classification: Python script / subsystem within Airflow DAG
- Definition: Cleans, parses, and validates GTFS files, handles missing/suspect values, and standardizes schema. Normalizes data into consistent structures for downstream storage and querying.
- Constraints:
 - Must support GTFS standard schema.
 - Must continue processing even if some records are incomplete.
 - Task must be completed within Airflow DAG execution window (Hourly)
- Resources:
 - Python libraries (pandas).
- Interface/Exports:
 - Input: Raw GTFS files exist from Module 1
 - Output: Cleaned and structured dataset ready for storage.
 - Exported to Module 3 (DuckDB)

Module 3: Data Storage

- Classification: Subsystem responsible for persistent storage of processed data, implemented as DuckDB database with Python interface.
- Definition: Stores processed GTFS data in relational format in DuckDB. Provides SQL query capability for usage downstream.
- Constraints:
 - VM storage: limited to 100 GB
 - Schema must remain constant across all pipeline runs
 - Required cleaned data from Module 2
 - Data must be written to DuckDB in proper format to be queried
- Resources:
 - Database: DuckDB
 - Local File Storage on VM (100 GB on disk)
 - Python DuckDB package for connection
 - CPU and disk I/O resources for insert queries
- Interface/Exports:
 - Input: Cleaned and Processed data from Module 2
 - Output: DuckDB tables
 - Exports: No planned exports. The system will have data available for future queries and analysis.

Scheduling: Apache Airflow

- Classification: Workflow scheduling managing DAGs and Python script execution.
- Definition: Schedules, monitors, and retried ETL tasks. Provides logging, alerting, and modular execution of Python modules.
- Constraints:
 - VM resources (CPU, RAM, Storage, I/O)
 - Tasks must be executed in order with logging and error handling
- Resources:
 - Requires Airflow scheduler and executor on VM
 - Log files for debugging and monitoring
- Interface/Exports:
 - Input: Defined DAG structure
 - Output: Task execution, logs, and error handling
 - Exports: No exports from Airflow, used for coordinating and scheduling tasks

Development

The development phase centered on constructing a reliable and efficient Extract, Transform, and Load (ETL) pipeline designed to process MARTA's General Transit Feed Specification (GTFS) data from start to finish. This pipeline was designed to ingest, prepare, and store MARTA's public transit data, enabling comprehensive analysis and insights.

The core of the system revolved around the ETL process. **Extract** involved retrieving GTFS data, which provides crucial schedule and vehicle information for MARTA's public transit operations. This encompasses data feeds containing details on routes, stops, schedules, and real-time vehicle positions. The **Transform** stage focused on data cleaning, standardization, and enrichment. This included handling missing values, resolving inconsistencies, converting data types, and potentially incorporating external datasets to enhance the available information. Finally, the **Load** phase involved storing the transformed data in a DuckDB database, an in-process, high-performance data management system optimized for analytical workloads.

To orchestrate and manage the complex workflow, Apache Airflow was implemented. Airflow provided robust control over the entire pipeline, ensuring that each task was executed in the correct order and dependencies were properly managed. This included tasks such as data extraction, data transformation, database interaction, and validation checks.

The development efforts primarily focused on several key areas:

Database Connection

DuckDB serves as the main storage engine. It handles large GTFS files with strong speed and stability. Early tests showed fast SQL performance, which helped shorten iteration time. The code opens and closes database sessions through safe context blocks. This avoids long open sessions, locked tables, and issues with stale connections. Read operations stay isolated while writing operations to protect the database from long term wear. The schema follows the standard GTFS layout. This reduced extra work and allowed the team to rely on documented field runs. Queries against the schema stayed stable through development.

Architecture Overview

The database connection architecture is built around DuckDB, an analytical database system optimized for processing MARTA's GTFS data (2.4M+ records). The database structure follows the GTFS specification with optimized tables for the static and real time data:

Static Data

- Core transit data (agency, routes, stops, trips)
- Schedule information (stop_times, calendar, calendar_dates)
- Geographic data (shapes for route mapping)
- Static GTFS data with monthly update frequency

Real Time Data

- Core data fields (entity_id, vehicle_id, trip_id, longitude, latitude, speed, current_stop_sequence)
- Eta/delay logic joins realtime rows to static trips and stops
- Distance computed with inline SQL Haversine
- Eta Calculated using the distance divided by the realtime speed

- Vehicle positions are updated every 5 to 30 seconds
- Upon ingestion vehicle_positions table is pushed to MotherDuck to where it can be accessed by our website.

GTFSProcessor

The GTFSProcessor is the brains of the database and has all methods and logic for validating and loading data into the DuckDB tables. Below is a list of functions that the Processor provides.

- Handling core database operations across GTFS tables
- Implementing data validation through coordinate boundary checks, time format standardization, and referential integrity verification
- Ensuring safe database connections with automatic resource cleanup
- Providing methods for bulk loading of static GTFS feeds and real-time vehicle positioning
- Maintaining referential integrity across transit system tables

GTFSManager

The GTFSManager is the interface for interacting with the Processor. Below is a list of functions that the Manager provides.

- A high-level interface for database operations
- Methods for transit system statistics and health checks
- Support for analytical SQL queries for route and schedule analysis

Testing

1. Test Objectives

- (1) Ensure retrieving of static GTFS file is successful.
- (2) Ensure static data is successfully preserved in DuckDB database.
- (3) Ensure vehicle and trips realtime files download successfully.
- (4) Ensure vehicle and trips data is parsed and cleaned.
- (5) Ensure vehicle data is successfully preserved in DuckDB database.
- (6) Ensure trips data is successfully preserved in DuckDB database.

(7) Ensure Apache Airflow properly manages repeated runs of the DAG.

2. Scope of Testing

- (1) In-scope: Preserving of static data for trips, routes, stops, vehicles
- (2) Out-of-scope: Extraneous GTFS static attributes – agency, fare_attributes, etc. Specifically, trip_times is out-of-scope as we found it to be much too unreliable.
- (3) In-scope: Preserving realtime data for vehicles and trips.
- (4) Out-of-scope: Realtime data for alerts, if present, will be out-of-scope.
- (5) In-scope: Airflow successfully running while setup and internet connection are working properly.
- (6) Out-of-scope: Issues with connectivity, or faulty setup interfering with Airflow will be out-of-scope.

3. Test Cases

3.1 Static file retrieval

- i. GTFS static zip file downloaded.
- ii. With the system properly set up, the GTFS static data zip file will be downloaded correctly.

3.2 Static data saved

- iii. Static data saved in DuckDB database.
- iv. After downloading the static zip file, the .txt files for trips, routes, stops, and vehicles are stored in the database in their respective tables.

3.3 Realtime files retrieved

- v. Realtime data retrieval.
- vi. With the system properly set up, the GTFS realtime data files will be downloaded correctly.

3.4 Realtime data saved

- vii. Realtime data saved in DuckDB database.
- viii. After downloading the .pb files for vehicles and trips, the data is stored in the database in their respective tables.

3.5 Realtime data processed

- ix. Realtime data for vehicles and trips processed

- x. Vehicle and trips data is parsed and cleaned properly, with entries containing missing or invalid fields removed.

3.6 Airflow managing tasks

- xi. The DAG is repeated by Airflow
- xii. Airflow runs the DAG successfully and repeats the runs at the desired interval, in this case, every hour.

4. Test Procedures

4.1 Static file retrieval

- xiii. Step 1: Execute a single run of the DAG.
- xiv. Step 2: Observe that ZIP file with static data was retrieved.

4.2 Static data saved

- xv. Step 1: Execute a single run of the DAG with the correct code.
- xvi. Step 2: Manually query the database to ensure that the static data were successfully preserved in the database.

4.3 Realtime files retrieved

- xvii. Step 1: Execute a single run of the DAG.
- xviii. Step 2: Observe that the .pb files for both trips and vehicles were successfully retrieved.

4.4 Realtime data saved

- xix. Step 1: Execute a single run of the DAG with the correct code.
- xx. Step 2: Manually query the database to ensure that the realtime data for both trips and vehicles were successfully preserved in the database.

4.5 Realtime data processed

- xxi. Step 1: Execute a single run of the DAG with the correct code.
- xxii. Step 2: Manually query the database to ensure that improper data was not saved, such as trips with irrational arrival_time data.

4.6 Airflow managing tasks

- xxiii. Step 1: Begin the DAG in Airflow.
- xxiv. Step 2: Wait for the amount of time designated as the DAG frequency.

- xxv. Step 3: Ensure that the DAG is repeated by Airflow at the proper frequency.

Test Environment

- (7) Hardware – Apple MacBook: Apple M1 Pro, 16 GB memory
- (8) Software – The software used to set up and run these tests are Python version 3.12.12 for the coding of the project and tests, Apache Airflow to run the DAG and to manage repeated runs, and the operating system the tests were performed on was MacOS 15.6.1.

Test Data

- (9) Static test data is from the MARTA static data feed.
- (10) Realtime data is from MARTA vehicles and trips realtime (.pb) feeds.

Software Test Report (STR)

Requirement	Pass	Fail	Severity
3.1 Static file retrieval	Pass		
3.2 Static data saved	Pass		
3.3 Realtime files retrieved	Pass		
3.4 Realtime files saved	Pass		
3.5 Realtime files processed	Pass		
3.6 Airflow managing Tasks	Pass		

Conclusion

This project set out to build a full ETL pipeline that handles both static and real time MARTA GTFS data in a structured and reliable way. The system now retrieves the data, processes it, and stores it in DuckDB with consistency. It uses Apache Airflow to manage tasks and enforce dependable scheduling. The work ran through several stages including research, design, development, testing, and repeated refinement. Each stage added more stability to the pipeline and shaped it into a tool that supports large data workloads.

There were many obstacles that were encountered as the project progressed. These ranged from the learning curve of using unfamiliar technologies like GTFS and Apache Airflow, to the realization that the data provided by the MARTA transit agency was not as trustworthy as first expected. At the end of the project, all of these obstacles were addressed. The

completed pipeline now handles all the required tasks and preserves the data in a clean and usable format for future analysis or additional data fields.

Appendix

Appendix A: GTFS extraction, transformation, and load code

Description: GTFS extraction, transformation, and load

```
def _paths():
    data_dir = Variable.get("DATA_DIR", default_var="/opt/airflow/include/data")
    db_path = Variable.get("DUCKDB_PATH", default_var="/opt/airflow/include/data/gtfs.duckdb")
    os.makedirs(data_dir, exist_ok=True)
    return data_dir, db_path
```

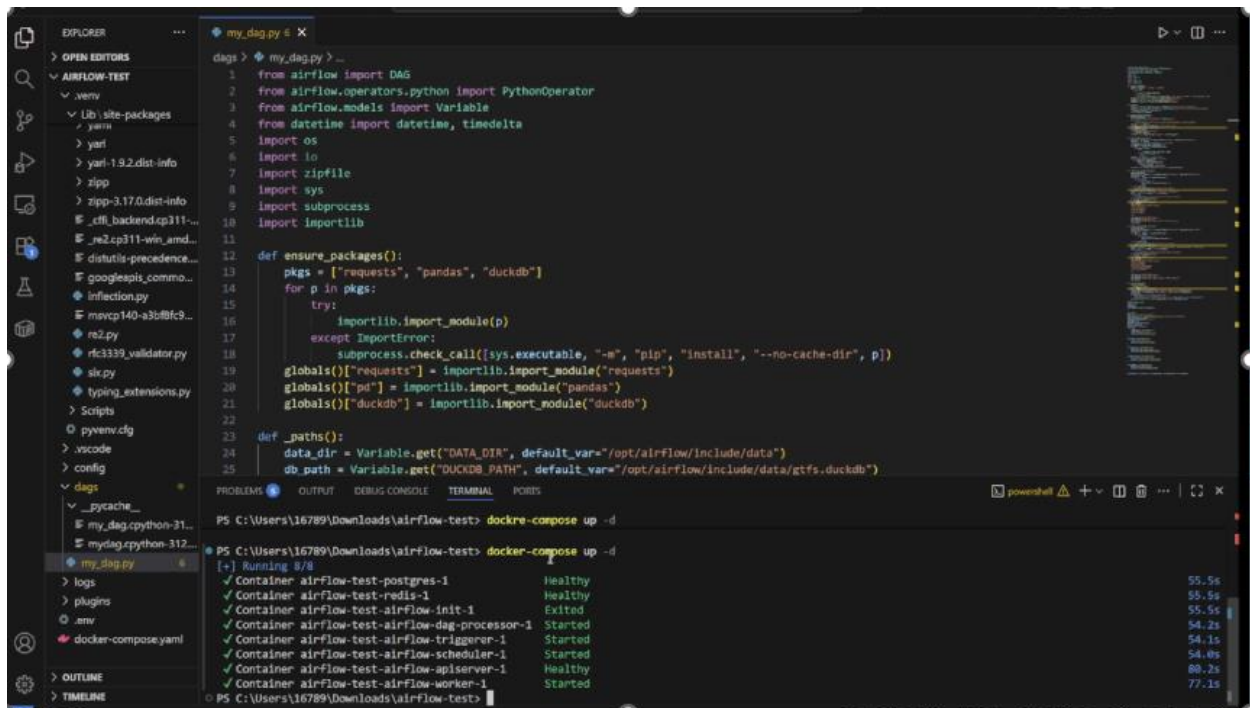
```
def extract_gtfs(**context):
    zip_path = context["ti"].xcom_pull(key="zip_path", task_ids="download_gtfs")
    data_dir, _ = _paths()
    extract_dir = os.path.join(data_dir, "extracted")
    os.makedirs(extract_dir, exist_ok=True)
    for root, _, files in os.walk(extract_dir):
        for name in files:
            try:
                os.remove(os.path.join(root, name))
            except FileNotFoundError:
                pass
    wanted = {"stops.txt", "routes.txt"}
    with zipfile.ZipFile(zip_path, "r") as z:
        for name in z.namelist():
            base = os.path.basename(name)
            if base in wanted:
                z.extract(name, extract_dir)
    context["ti"].xcom_push(key="extract_dir", value=extract_dir)
```

```

def load_stops_duckdb(**context):
    ensure_packages()
    extract_dir = context["ti"].xcom_pull(key="extract_dir", task_ids="extract_gtfs")
    stops_file = None
    for root, _, files in os.walk(extract_dir):
        for f in files:
            if f == "stops.txt":
                stops_file = os.path.join(root, f)
                break
    if not stops_file:
        raise FileNotFoundError("stops.txt not found")
    df = pd.read_csv(stops_file)
    keep = ["stop_id", "stop_name", "stop_lat", "stop_lon"]
    cols = [c for c in keep if c in df.columns]
    if not cols:
        raise ValueError("stops.txt missing required columns")
    df = df[cols].drop_duplicates()
    _, db_path = _paths()
    con = duckdb.connect(db_path)
    con.execute("""
CREATE TABLE IF NOT EXISTS stops (
    stop_id VARCHAR,
    stop_name VARCHAR,
    stop_lat DOUBLE,
    stop_lon DOUBLE
)
""")
    con.execute("DELETE FROM stops")
    con.register("stops_df", df)
    con.execute("INSERT INTO stops SELECT * FROM stops_df")
    con.close()

```


Appendix B: Airflow DAG setup



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal window at the bottom. The code editor displays a Python script named `my_dag.py` with the following content:

```
1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3 from airflow.models import Variable
4 from datetime import datetime, timedelta
5 import os
6 import io
7 import zipfile
8 import sys
9 import subprocess
10 import importlib
11
12 def ensure_packages():
13     pkgs = ["requests", "pandas", "duckdb"]
14     for p in pkgs:
15         try:
16             importlib.import_module(p)
17         except ImportError:
18             subprocess.check_call([sys.executable, "-m", "pip", "install", "--no-cache-dir", p])
19     globals()["requests"] = importlib.import_module("requests")
20     globals()["pd"] = importlib.import_module("pandas")
21     globals()["duckdb"] = importlib.import_module("duckdb")
22
23 def _paths():
24     data_dir = Variable.get("DATA_DIR", default_var="/opt/airflow/include/data")
25     db_path = Variable.get("DUCKDB_PATH", default_var="/opt/airflow/include/data/gtfs.duckdb")
```

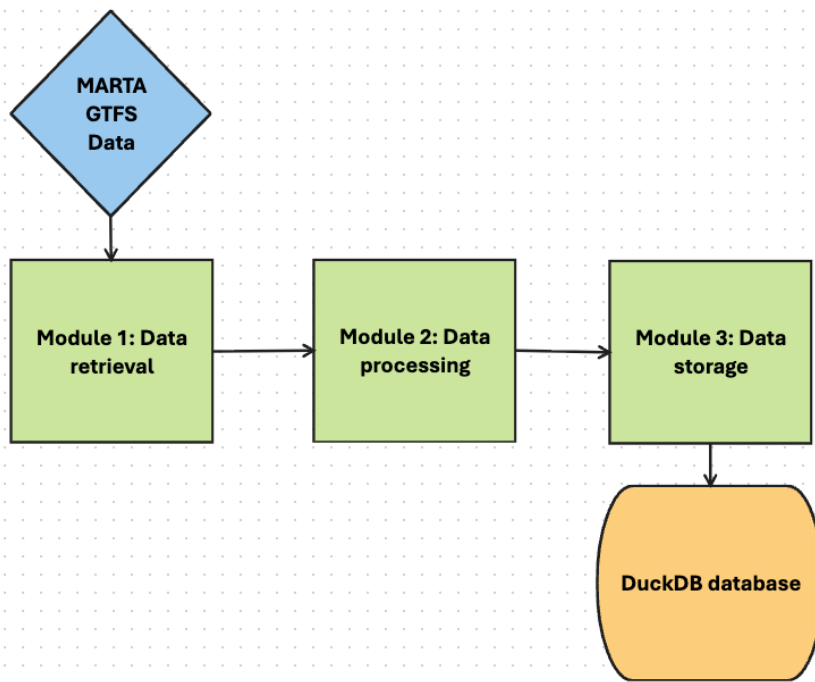
The terminal window shows the output of the `docker-compose up -d` command, indicating that the containers are running successfully:

```
PS C:\Users\16789\Downloads\airflow-test> docker-compose up -d
[+] Running 8/8
✓ Container airflow-test-postgres-1 Healthy 55.5s
✓ Container airflow-test-redis-1 Healthy 55.5s
✓ Container airflow-test-airflow-init-1 Exited 55.5s
✓ Container airflow-test-airflow-dag-processor-1 Started 54.2s
✓ Container airflow-test-airflow-triggerer-1 Started 54.1s
✓ Container airflow-test-airflow-scheduler-1 Started 54.0s
✓ Container airflow-test-airflow-apiserver-1 Healthy 80.2s
✓ Container airflow-test-airflow-worker-1 Started 77.1s
```

Description: Airflow setup code

Appendix C: ETL Pipeline data

Logical View of Data Flow



Description: The ETL pipeline data flow starting with MARTA's GTFS data.